

Name: Gaurika Mittal

Batch: 15

Sap Id: 590034892

Experiment-15

Question

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, with **void return type** and with a **simple function call without passing arguments**. Each function should read the required input, process it, and display the result.

The menu should perform the following operations:

1. Check whether the digits are in increasing order from left to right.
2. Check whether the digits are in decreasing order from left to right.
3. Find the largest digit and the smallest digit.
4. Find the second largest digit and second smallest digit.

1. Algorithm

Main Program Algorithm

1. Start.
2. Declare an integer variable `choice`.
3. Display the four menu options.
4. Input the user's choice.
5. Use a **switch-case statement**.
6. If choice is 1, call `increasing()`.
7. If choice is 2, call `decreasing()`.
8. If choice is 3, call `largestSmallest()`.
9. If choice is 4, call `secondLargestSmallest()`.
10. For any other choice, display "Invalid choice".
11. Stop.

Function 1: `increasing()`

1. Start the function.
2. Input a positive integer.
3. Reverse the number so that its digits can be checked from left to right.
4. Take the first digit as the previous digit.
5. Compare every next digit with the previous digit.

6. If a digit is less than or equal to the previous digit, set the flag to 0.
7. Otherwise, continue checking the remaining digits.
8. If the flag remains 1, display "Digits are in increasing order".
9. Otherwise, display "Digits are not in increasing order".
10. End the function.

Function 2: decreasing()

1. Start the function.
2. Input a positive integer.
3. Reverse the number so that its digits can be checked from left to right.
4. Take the first digit as the previous digit.
5. Compare every next digit with the previous digit.
6. If a digit is greater than or equal to the previous digit, set the flag to 0.
7. Otherwise, continue checking the remaining digits.
8. If the flag remains 1, display "Digits are in decreasing order".
9. Otherwise, display "Digits are not in decreasing order".
10. End the function.

Function 3: largestSmallest()

1. Start the function.
2. Input a positive integer.
3. Initialize `largest = 0` and `smallest = 9`.
4. Extract each digit using `% 10`.
5. Compare the digit with `largest`. If it is greater, update `largest`.
6. Compare the digit with `smallest`. If it is smaller, update `smallest`.
7. Remove the last digit using `/ 10`.
8. Repeat until all digits have been checked.
9. Display the largest and smallest digits.
10. End the function.

Function 4: secondLargestSmallest()

1. Start the function.
2. Input a positive integer.
3. Create a frequency array for digits 0 to 9.
4. Extract each digit of the number.
5. Mark the digit as present in the frequency array.
6. Search from 0 to 9 to find the second smallest distinct digit.
7. Search from 9 to 0 to find the second largest distinct digit.
8. If they exist, display the second largest and second smallest digits.
9. Otherwise, display that they do not exist.
10. End the function.

2. Test Case Table

Test Case	Choice	Inputs	Expected Output	Actual Output	Results
1	1	12359	Digits are in increasing order	Digits are in increasing order	Pass
2	1	12435	Digits are not in increasing order	Digits are not in increasing order	Pass
3	2	97531	Digits are in decreasing order	Digits are in decreasing order	Pass
4	2	97351	Digits are not in decreasing order	Digits are not in decreasing order	Pass
5	3	50728	Largest digit = 8, Smallest digit = 0	Largest digit = 8, Smallest digit = 0	Pass
6	4	58321	Second largest digit = 5, Second smallest digit = 2	Second largest digit = 5, Second smallest digit = 2	Pass
7	4	7777	Second largest and second smallest digits do not exist	Second largest and second smallest digits do not exist	Pass

3. C Program

```
#include <stdio.h>
void increasing(void);
void decreasing(void);
void largestSmallest(void);
void secondLargestSmallest(void);
int main(void)
{
    int choice;
    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");
```

```

printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
    case 1:
        increasing();
        break;
    case 2:
        decreasing();
        break;
    case 3:
        largestSmallest();
        break;
    case 4:
        secondLargestSmallest();
        break;
    default:
        printf("Invalid choice\n");
}
return 0;
}
void increasing(void)
{
    long long n, rev = 0, temp;
    int prev, digit, flag = 1;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    temp = n;
    while (temp > 0)
    {
        rev = rev * 10 + temp % 10;
        temp = temp / 10;
    }
    prev = rev % 10;
    rev = rev / 10;
    while (rev > 0)
    {
        digit = rev % 10;
        if (digit <= prev)
        {
            flag = 0;
            break;
        }
        prev = digit;
        rev = rev / 10;
    }
    if (flag == 1)
        printf("Digits are in increasing order\n");
    else
        printf("Digits are not in increasing order\n");
}
void decreasing(void)
{
    long long n, rev = 0, temp;
    int prev, digit, flag = 1;

```

```

printf("Enter a positive integer: ");
scanf("%lld", &n);
temp = n;
while (temp > 0)
{
    rev = rev * 10 + temp % 10;
    temp = temp / 10;
}
prev = rev % 10;
rev = rev / 10;
while (rev > 0)
{
    digit = rev % 10;
    if (digit >= prev)
    {
        flag = 0;
        break;
    }
    prev = digit;
    rev = rev / 10;
}
if (flag == 1)
    printf("Digits are in decreasing order\n");
else
    printf("Digits are not in decreasing order\n");
}

void largestSmallest(void)
{
    long long n;
    int digit, largest = 0, smallest = 9;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    while (n > 0)
    {
        digit = n % 10;
        if (digit > largest)
            largest = digit;
        if (digit < smallest)
            smallest = digit;
        n = n / 10;
    }
    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

void secondLargestSmallest(void)
{
    long long n;
    int digit, i;
    int freq[10] = {0};
    int secondSmallest = -1;
    int secondLargest = -1;
    int count = 0;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

```

```

while (n > 0)
{
    digit = n % 10;
    freq[digit] = 1;
    n = n / 10;
}
for (i = 0; i <= 9; i++)
{
    if (freq[i] == 1)
    {
        count++;
        if (count == 2)
        {
            secondSmallest = i;
            break;
        }
    }
}
count = 0;
for (i = 9; i >= 0; i--)
{
    if (freq[i] == 1)
    {
        count++;

        if (count == 2)
        {
            secondLargest = i;
            break;
        }
    }
}
if (secondSmallest == -1 || secondLargest == -1)
{
    printf("Second largest and second smallest digits do not exist\n");
}
else
{
    printf("Second largest digit = %d\n", secondLargest);
    printf("Second smallest digit = %d\n", secondSmallest);
}
}

```

4.Compilation and Execution

```
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 1  
Enter a positive integer: 12359  
Digits are in increasing order  
  
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 1  
Enter a positive integer: 12435  
Digits are not in increasing order
```

```
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 2  
Enter a positive integer: 97531  
Digits are in decreasing order  
  
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 2  
Enter a positive integer: 97351  
Digits are not in decreasing order
```

```
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 3  
Enter a positive integer: 50728  
Largest digit = 8  
Smallest digit = 0
```

```
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 4  
Enter a positive integer: 58321  
Second largest digit = 5  
Second smallest digit = 2
```

```
(kali㉿kali)-[~]  
$ gcc hdoc16.c -o hdoc16  
  
(kali㉿kali)-[~]  
$ ./hdoc16  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 4  
Enter a positive integer: 7777  
Second largest and second smallest digits do not exist
```